

App package requirements for MSIX app - Windows apps

Follow these guidelines to prepare your app's packages for submission to the Microsoft Store.

Make sure to [test your app with the Windows App Certification Kit](#). We also recommend that you test your app on different types of hardware. Note that until we certify your app and make it available from the Microsoft Store, it can only be installed and run on computers that have developer licenses.

If you're using Microsoft Visual Studio as your development environment, you already have built-in tools that make creating an app package a quick and easy process. For more info, see [Packaging apps](#).

Note

Be sure that all your filenames use ANSI.

When you create your package in Visual Studio, make sure you are signed in with the same account associated with your developer account. Some parts of the package manifest have specific details related to your account. This info is detected and added automatically. Without the additional information added to the manifest, you may encounter package upload failures.

- **App Package (.msix or .appx):** A single package that contains your application and its resources, targeted at a single device architecture. For example, an x64 or x86 application package. To target multiple architectures with an app bundle you'd need to generate one for each architecture.
- **App Bundle (.msixbundle or .appxbundle):** An app bundle is a type of package that can contain multiple app packages, each of which is built to support a specific device architecture. For example, an

app bundle can contain three separate app packages for the x86, x64, and ARM configurations. App bundles should be generated whenever possible because they allow your app to be available on the widest possible range of devices.

- **App Package Upload File (.msixupload or .appxupload) - for Store Submission only:** A single file that can contain multiple app packages or an app bundle to support various processor architectures. The app package upload file also contains a symbol file (an .appxsym file) to evaluate [crash analytics](#) after your app that has been published in the Microsoft Store. This file will be automatically created for you if you are packaging your app with Visual Studio with the intention of submitting it to Partner Center for publishing to the Microsoft Store. **Note:** An appxsym file is a compressed .pdb file containing public symbols of your app used for [crash analytics](#) in Partner Center. You can omit this file, but if you do, no crash analytic or debugging information will be available for your app.

When you build your app's UWP packages, Visual Studio can create an .msix or appx file, or a .msixupload or .appxupload file. For UWP apps, we recommend that you always upload the .msixupload or .appxupload file in the [Packages](#) page. For more info about packaging UWP apps for the Store, see [Package a UWP app with Visual Studio](#).

Your MSIX and AppX packages don't have to be signed with a certificate rooted in a trusted certificate authority when submitting to the Microsoft Store. The Microsoft Store will automatically re-sign your MSIX/AppX packages with a Microsoft certificate during the publishing process after your app passes certification. This means:

- You don't need to purchase a CA-trusted code signing certificate for MSIX/AppX Store submissions
- You don't need to provide a .pfx or .cer file from a certificate authority to submit MSIX/AppX packages
- USB tokens or hardware security modules (HSMs) are not required for MSIX/AppX Store submissions
- The Store replaces any existing signature on MSIX/AppX packages with a Microsoft certificate, providing trust and security to customers

Note

If you are submitting an **MSI or EXE installer** to the Store, the Store does not re-sign those files. You must Authenticode-sign your MSI/EXE installer yourself with a valid code signing certificate before submission.

Note

If you are distributing your MSIX package outside the Microsoft Store (for example, for enterprise deployment or sideloading), you will need to sign the package yourself with your own code signing certificate. For more information, see [Sign an app package using SignTool](#).

If you don't use Visual Studio to create your package, you must [create your package manifest manually](#).

Be sure to review the [App package manifest](#) documentation for complete manifest details and requirements. Your manifest must follow the package manifest schema in order to pass certification.

Your manifest must include some specific info about your account and your app. You can find this info by looking at [View app identity details](#) in the **Product management** section of your app's overview page in the dashboard.

Note

Values in the manifest are case-sensitive. Spaces and other punctuation must also match. Enter the values carefully and review them to ensure that they are correct.

App bundles (.msixbundle or .appxbundle) use a different manifest. Review the [Bundle manifest](#) documentation for the details and requirements for app bundle manifests. Note that in a .msixbundle or .appxbundle, the manifest of each included package must use the same elements and attributes, except for the **ProcessorArchitecture** attribute of the [Identity](#) element.

Tip

Be sure to run the [Windows App Certification Kit](#) before you submit your packages. This can help you determine if your manifest has any problems that might cause certification or submission failures.

Your app's packages must comply with these requirements.

App package property	Requirement
Package size	.msixbundle or .appxbundle: 25 GB maximum per bundle .msix or .appx packages targeting Windows 10 or Windows 11: 25 GB maximum per package
Block map hashes	SHA2-256 algorithm

For UWP apps, all packages must target a version of Windows 10 or Windows 11 supported by the Store. The versions your package supports must be indicated in the **MinVersion** and **MaxVersionTested** attributes of the [TargetDeviceFamily](#) element of the app manifest.

StoreManifest.xml is an optional configuration file that may be included in app packages. Its purpose is to enable features, such as declaring your app as a Microsoft Store device app or declaring requirements that a package depends on to be applicable to a device, that the package manifest does not cover. If used, StoreManifest.xml is submitted with the app package and must be in the root folder of your app's main project. For more info, see [StoreManifest schema](#).

Each package you provide must have a version number (provided as a value in the **Version** attribute of the [Package/Identity](#) element in the app manifest). The Microsoft Store enforces certain rules related to version numbers, which work somewhat differently in different OS versions.

Note

This topic refers to "packages", but unless noted, the same rules apply to version numbers for both .msix/.appx and

.msixbundle/.appxbundle files.

Important

For Windows 10 or Windows 11 (UWP) packages, the last (fourth) section of the version number is reserved for Store use and must be left as 0 when you build your package (although the Store may change the value in this section). The other sections must be set to an integer between 0 and 65535 (except for the first section, which cannot be 0).

When choosing a UWP package from your published submission, the Microsoft Store will always use the highest-versioned package that is applicable to the customer's Windows 10 or Windows 11 device. This gives you greater flexibility and puts you in control over which packages will be provided to customers on specific device types. Importantly, you can submit these packages in any order; you are not limited to providing higher-versioned packages with each subsequent submission.

You can provide multiple UWP packages with the same version number. However, packages that share a version number cannot also have the same architecture, because the full identity that the Store uses for each of your packages must be unique. For more info, see [Identity](#).

When you provide multiple UWP packages that use the same version number, the architecture (in the order x64, x86, Arm, neutral) will be used to decide which one is of higher rank (when the Store determines which package to provide to a customer's device). When ranking app bundles that use the same version number, the highest architecture rank within the bundle is considered: an app bundle that contains an x64 package will have a higher rank than one that only contains an x86 package.

This gives you a lot of flexibility to evolve your app over time. You can upload and submit new packages that use lower version numbers to add support for Windows 10 or Windows 11 devices that you did not previously support, you can add higher-versioned packages that have stricter dependencies to take advantage of hardware or OS features,

or you can add higher-versioned packages that serve as updates to some or all of your existing customer base.

The following example illustrates how version numbering can be managed to deliver the intended packages to your customers over multiple submissions.

Windows 10 enables you to write a single codebase that runs everywhere. This makes starting a new cross-platform project much easier. However, for a number of reasons, you might not want to merge existing codebases to create a single project right away.

You can use the package versioning rules to gradually move your customers to a single package for the Universal device family, while shipping a number of interim updates for specific device families (including ones that take advantage of Windows 10 APIs). The example below illustrates how the same rules are consistently applied over a series of submissions for the same app.

Submission	Contents	Customer experience
1	- Package version: 1.1.10.0 - Device family: Windows.Desktop, minVersion 10.0.10240.0	- Devices on Windows 10 and 11 Desktop build 10.0.10240.0 and above will get 1.1.10.0 - Other device families will not be able to purchase and install the app
2	- Package version: 1.1.10.0 - Device family: Windows.Desktop, minVersion 10.0.10240.0 - Package version: 1.0.0.0 - Device family: Windows.Universal,	- Devices on Windows 10 and 11 Desktop build 10.0.10240.0 and above will get 1.1.10.0 - Other (non-desktop) device families when they are introduced will get 1.0.0.0 - Desktop devices that already have the app installed will not see any update (because they

Submission	Contents	Customer experience
	minVersion 10.0.10240.0	already have the best available version 1.1.10.0 and are higher than 1.0.0.0)
3	- Package version: 1.1.10.0 - Device family: Windows.Desktop, minVersion 10.0.10240.0 - Package version: 1.1.5.0 - Device family: Windows.Universal, minVersion 10.0.10250.0 - Package version: 1.0.0.0 - Device family: Windows.Universal, minVersion 10.0.10240.0	- Devices on Windows 10 and 11 Desktop build 10.0.10240.0 and above will get 1.1.10.0 - Other (non-desktop) device families when introduced with build 10.0.10250.0 and above will get 1.1.5.0 - Other (non-desktop) device families when introduced with build $\geq 10.0.10240.0$ and $< 10.0.10250.0$ will get 1.1.0.0 - Desktop devices that already have the app installed will not see any update (because they already have the best available version 1.1.10.0 which is higher than both 1.1.5.0 and 1.0.0.0)
4	- Package version: 2.0.0.0 - Device family: Windows.Universal, minVersion 10.0.10240.0	- All customers on all device families on Windows 10 and 11 build $\geq 10.0.10240.0$ and above will get package 2.0.0.0

Note

In all cases, customer devices will receive the package that has the highest possible version number that they qualify for. For example, in

the third submission above, all desktop devices will get v1.1.10.0, even if they have OS version 10.0.10250.0 or later and thus could also accept v1.1.5.0. Since 1.1.10.0 is the highest version number available to them, that is the package they will get.

If you keep copies of your packages, you'll have the option to roll back your app's package in the Store to an earlier Windows 10 package if you should discover problems with a release. This is a temporary way to limit the disruption to your customers while you take time to fix the issue.

To do this, create a new [submission](#). Remove the problematic package and upload the old package that you want to provide in the Store. Customers who have already received the package you are rolling back will still have the problematic package (since your older package will have an earlier version number). But this will stop anyone else from acquiring the problematic package, while allowing the app to still be available in the Store.

To fix the issue for the customers who have already received the problematic package, you can submit a new Windows 10 package that has a higher version number than the bad package as soon as you can. After that submission goes through the certification process, all customers will be updated to the new package, since it will have a higher version number.

You can submit apps to the Microsoft Store in over 100 languages.

To learn more about configuring languages in your apps, see [Globalization and localization](#) and [Understand user profile languages and app manifest languages](#). We also have a [Multilingual App Toolkit](#) to help you write apps that support multiple languages.

These are the languages that the Microsoft Store supports. Your app must support at least one of these languages.

Language codes that are not included here are not supported by the Store. We recommend that you don't include packages targeting language codes other than those listed below; such packages will not

be distributed to customers, and may cause delays or failures in certification.

Language name	Supported language codes
Arabic	ar, ar-sa, ar-ae, ar-bh, ar-dz, ar-eg, ar-iq, ar-jo, ar-kw, ar-lb, ar-ly, ar-ma, ar-om, ar-qa, ar-sy, ar-tn, ar-ye
Afrikaans	af, af-za
Albanian	sq, sq-al
Amharic	am, am-et
Armenian	hy, hy-am
Assamese	as, as-in
Azerbaijani	az-arab, az-arab-az, az-cyrl, az-cyrl-az, az-latn, az-latn-az
Basque (Basque)	eu, eu-es
Belarusian	be, be-by
Bangla	bn, bn-bd, bn-in
Bosnian	bs, bs-cyrl, bs-cyrl-ba, bs-latn, bs-latn-ba
Bulgarian	bg, bg-bg
Catalan	ca, ca-es, ca-es-valencia
Cherokee	chr-cher, chr-cher-us, chr-latn
Chinese (Simplified)	zh-Hans, zh-cn, zh-hans-cn, zh-sg, zh-hans-sg
Chinese (Traditional)	zh-Hant, zh-hk, zh-mo, zh-tw, zh-hant-hk, zh-hant-mo, zh-hant-tw
Croatian	hr, hr-hr, hr-ba

Language name	Supported language codes
Czech	cs, cs-cz
Danish	da, da-dk
Dari	prs, prs-af, prs-arab
Dutch	nl, nl-nl, nl-be
English	en, en-au, en-ca, en-gb, en-ie, en-in, en-nz, en-sg, en-us, en-za, en-bz, en-hk, en-id, en-jm, en-kz, en-mt, en-my, en-ph, en-pk, en-tt, en-vn, en-zw, en-053, en-021, en-029, en-011, en-018, en-014
Estonian	et, et-ee
Filipino	fil, fil-latn, fil-ph
Finnish	fi, fi-fi
French	fr, fr-be , fr-ca , fr-ch , fr-fr , fr-lu, fr-015, fr-cd, fr-ci, fr-cm, fr-ht, fr-ma, fr-mc, fr-ml, fr-re, frc-latn, frp-latn, fr-155, fr-029, fr-021, fr-011
Galician	gl, gl-es
Georgian	ka, ka-ge
German	de, de-at, de-ch, de-de, de-lu, de-li
Greek	el, el-gr
Gujarati	gu, gu-in
Hausa	ha, ha-latn, ha-latn-ng
Hebrew	he, he-il
Hindi	hi, hi-in
Hungarian	hu, hu-hu
Icelandic	is, is-is

Language name	Supported language codes
Igbo	ig-latn, ig-ng
Indonesian	id, id-id
Inuktitut (Latin)	iu-cans, iu-latn, iu-latn-ca
Irish	ga, ga-ie
isiXhosa	xh, xh-za
isiZulu	zu, zu-za
Italian	it, it-it, it-ch
Japanese	ja , ja-jp
Kannada	kn, kn-in
Kazakh	kk, kk-kz
Khmer	km, km-kh
K'iche'	quc-latn, qut-gt, qut-latn
Kinyarwanda	rw, rw-rw
KiSwahili	sw, sw-ke
Konkani	kok, kok-in
Korean	ko, ko-kr
Kurdish	ku-arab, ku-arab-iq
Kyrgyz	ky-kg, ky-cyrl
Lao	lo, lo-la
Latvian	lv, lv-lv
Lithuanian	lt, lt-lt
Luxembourgish	lb, lb-lu
Macedonian	mk, mk-mk

Language name	Supported language codes
Malay	ms, ms-bn, ms-my
Malayalam	ml, ml-in
Maltese	mt, mt-mt
Maori	mi, mi-latn, mi-nz
Marathi	mr, mr-in
Mongolian (Cyrillic)	mn-cyrl, mn-mong, mn-mn, mn-phag
Nepali	ne, ne-np
Norwegian	nb, nb-no, nn, nn-no, no, no-no,
Odia	or, or-in
Persian	fa, fa-ir
Polish	pl, pl-pl
Portuguese (Brazil)	pt-br
Portuguese (Portugal)	pt, pt-pt
Punjabi	pa, pa-arab, pa-arab-pk, pa-deva, pa-in
Quechua	quz, quz-bo, quz-ec, quz-pe
Romanian	ro, ro-ro
Russian	ru , ru-ru
Scottish Gaelic	gd-gb, gd-latn
Serbian (Latin)	sr-Latn, sr-latn-cs, sr, sr-latn-ba, sr-latn-me, sr-latn-rs
Serbian (Cyrillic)	sr-cyrl, sr-cyrl-ba, sr-cyrl-cs, sr-cyrl-me, sr-cyrl-rs

Language name	Supported language codes
Sesotho sa Leboa	nso, nso-za
Setswana	tn, tn-bw, tn-za
Sindhi	sd-arab, sd-arab-pk, sd-deva
Sinhala	si, si-lk
Slovak	sk, sk-sk
Slovenian	sl, sl-si
Spanish	es, es-cl, es-co, es-es, es-mx, es-ar, es-bo, es-cr, es-do, es-ec, es-gt, es-hn, es-ni, es-pa, es-pe, es-pr, es-py, es-sv, es-us, es-uy, es-ve, es-019, es-419
Swedish	sv, sv-se, sv-fi
Tajik (Cyrillic)	tg-arab, tg-cyrl, tg-cyrl-tj, tg-latn
Tamil	ta, ta-in
Tatar	tt-arab, tt-cyrl, tt-latn, tt-ru
Telugu	te, te-in
Thai	th, th-th
Tigrinya	ti, ti-et
Turkish	tr, tr-tr
Turkmen	tk-cyrl, tk-latn, tk-tm, tk-latn-tr, tk-cyrl-tr
Ukrainian	uk, uk-ua
Urdu	ur, ur-pk
Uyghur	ug-arab, ug-cn, ug-cyrl, ug-latn
Uzbek (Latin)	uz, uz-cyrl, uz-latn, uz-latn-uz

Language name	Supported language codes
Vietnamese	vi, vi-vn
Welsh	cy, cy-gb
Wolof	wo, wo-sn
Yoruba	yo-latn, yo-ng

1. Do I need to package my app as an MSIX, or can I submit a traditional EXE/MSI installer?

The Microsoft Store supports multiple packaging formats. MSIX is the recommended format, but traditional EXE/MSI installers are also accepted.

Supported MSIX and related package formats:

- .msix
- .msixbundle
- .msixupload
- .appx
- .appxbundle
- .appxupload

`.xap` is a legacy package type associated with previously published apps and isn't used for new submissions.

MSIX benefits include:

- Free Microsoft code signing and CDN hosting.
- Easier updates and better integration with Windows features.
- Support for advanced capabilities like flighting and commerce.

Using the MSIX package format ensures a more reliable, secure, and streamlined installation and update experience for users.

EXE/MSI submission:

- Allowed since June 2021.

- You must provide a package URL for the installer as part of the submission; the installer must be hosted on your own infrastructure or CDN.
- Requirements:
 - Must be .exe or .msi only.
 - Offline installer – no downloads during setup.
 - Installer must not change after submission or bundle unrelated software.

Both app types can be submitted in Store depending on developer's needs.